

Infix to Postfix Conversion & Recursion

Q. Discuss the application of Stacks in evaluating mathematical expressions. Explain the process of Infix to Postfix conversion, Postfix evaluation, and how stacks handle Recursion.

****Definition to Remember****

****Infix notation****
- no parentheses
Stacks are used for

1. Operator Precedence (for conversion)

Operator	Precedence
----------	------------

Operator	Precedence
----------	------------

2. Infix to Postfix Conversion Algorithm

1. Scan expression left to right.

2. Oper

Example: `A \* (B + C)`

Symbol	Stack	Output
--------	-------	--------

Symbol	Stack	Output
--------	-------	--------

3. Postfix Evaluation Algorithm

1. Scan postfix string left to right.

2. Oper

Example: `5 3 + 2 \*` -> $(5+3)*2 = 16$

4. Recursion and the Call Stack

* When a function calls itself, the OS uses the ****Call Stack**** to manage it.

* Each

factorial(3)

factorial(3) -> 3 <- top (most recent call)

factoria

*****Must-Write Points (for 10 marks)*****

1. Infix =

*****Quick Recall*****

`Infix->P
compute